Ecole Supérieure
d'Informatique et du Numérique
COLLEGE OF ENGINEERING & ARCHITECTURE

COMPTE RENDU

SDN & Network Softwarization — TP4 — Installation manuelle de flux OpenFlow dans Open vSwitch (OVS)
4e année Cybersécurité — École Supérieure d'Informatique et du Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha

Filière : Cybersécurité

Année : 2025/2026

Enseignant : M. BOUHORMA

Date : 20 octobre 2026

Table des matières

Cadre du TP **2**

1 Environnement de travail **3**
1.1 Initialisation 3

2 Exemple 1 — Deux flux pour faire communiquer h1 et h2 **5**

3 Exemple 2 — Autoriser uniquement ICMP entre h1 et h2 **7**
3.1 Étapes 1 à 4 : installation des règles 7
3.2 Étape 5 : tests 8

4 Exemple 3 — Modifier puis supprimer un flux **9**
4.1 Visualiser les flux 9
4.2 Modifier une règle pour bloquer ICMP 10
4.3 Rétablir la communication 11
4.4 Supprimer uniquement cette règle 11

5 Exemple 4 — Réaliser une ACL avec OpenFlow **12**
5.1 Étapes 1 et 2 : réinitialisation et règle de base 12
5.2 Étape 3 : règle de blocage prioritaire 12
5.3 Vérification 13

6 Exemple 5 — Autoriser uniquement un service HTTP sur le port TCP 8000 **14**
6.1 Installation des règles 14
6.2 Démarrage du serveur et tests 15

7 Exemple 6 — Gérer la durée de vie des flux **16**

8 Synthèse des commandes OVS **18**

9 Observation avec Wireshark **18**

10 Synthèse **19**

11 Grille de validation rapide **20**

Cadre du TP

Plateforme	Outils	Mode
Ubuntu 26.04 LTS (portable `Taha-inspiron-16`)	Mininet, Open vSwitch (`ovs-vsctl`, `ovs-ofctl`), OpenFlow 1.3, Wireshark	Individuel
Approche	**Switch / contrôleur**	**Livrable**
Flux OpenFlow installés à la main avec `ovs-ofctl`, six ateliers progressifs	`OVSSwitch` (`ovsk`) en OpenFlow 1.3, aucun contrôleur SDN	Compte rendu avec captures

TABLE 1 – Cadre général du TP

Question fil rouge : *comment un switch Open vSwitch prend-il ses décisions de transfert sans contrôleur SDN, uniquement à partir des règles installées manuellement avec `ovs-ofctl` ?*

Objectifs :

— créer la topologie à un switch et trois hôtes, puis vérifier les ports OpenFlow avec `ovs-ofctl show` ;

— installer, modifier et supprimer des flux avec `add-flow`, `mod-flows` et `del-flows` ;

— écrire des règles qui filtrent sur les ports d'entrée, sur ARP, ICMP, IPv4 et sur les ports TCP ;

— réaliser une ACL (liste de contrôle d'accès) avec OpenFlow et la comparer aux ACL Cisco ;

— maîtriser la durée de vie d'un flux (`idle_timeout`, `hard_timeout`) et son identification par `cookie` ;

— observer le trafic avec Wireshark sur les interfaces du switch.

Règle commune à tout le TP :

```
--switch ovsk,protocols=OpenFlow13 --controller none
```

Le switch `s1` parle OpenFlow 1.3 et aucun contrôleur n'est lancé : seules les règles que nous installons servent au transfert. Toutes les commandes `ovs-ofctl` portent l'option `-O OpenFlow13`.

Règle de travail :

— chaque exemple repart d'une table de flux vide (`del-flows s1`), sauf l'exemple 3 qui reprend la table de l'exemple 2 ;

— ne pas se limiter à « le ping marche ou non » : relever ce que le switch a réellement fait, à partir des messages du ping et des compteurs de `dump-flows`.

Note sur l'environnement. Toutes les manipulations ont été faites directement sur mon portable Ubuntu (`Taha-inspiron-16`), dans le dossier `~/Bureau/UIR/4A/S7/SDN/TP4`, avec deux terminaux : le premier pour Mininet, le second pour `ovs-vsctl` et `ovs-ofctl`. Les commandes du sujet coupées par un \ en fin de ligne sont saisies sur une seule ligne : le \ n'est qu'une césure d'affichage du sujet.

1 Environnement de travail

La même topologie sert pour tous les exemples : un switch `s1` et trois hôtes. Les adresses MAC sont fixées par l'option `--mac`.

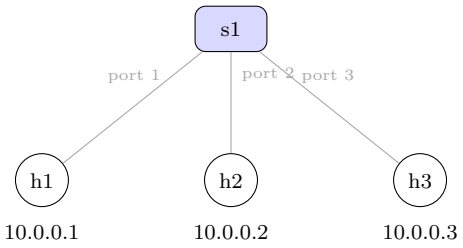


FIGURE 1 – Topologie du TP : `single,3`

Équipement	Adresse IP	Port OVS attendu
h1	10.0.0.1	s1-eth1 / 1
h2	10.0.0.2	s1-eth2 / 2
h3	10.0.0.3	s1-eth3 / 3

TABLE 2 – Plan d'adressage et ports attendus

1.1 Initialisation

Listing 1 – Terminal 1 : nettoyage puis création du réseau

```
sudo mn -c
sudo mn --topo single,3 --mac --switch ovsk,protocols=OpenFlow13
   --controller none
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP4$ sudo mn -c
*** Removing excess controllers/ofprotocols/ofdatapaths/pings/noxes
killall controller ofprotocol ofdatapath ping nox_corelt-nox_core ovs-openflowd ovs-controllerovs-testcontroller udpbwtest mnexec ivs ryu-manager 2> /dev/null
killall -9 controller ofprotocol ofdatapath ping nox_corelt-nox_core ovs-openflowd ovs-controllerovs-testcontroller udpbwtest mnexec ivs ryu-manager 2> /dev/null
pkill -9 -f "sudo mnexec"
*** Removing junk from /tmp
rm -f /tmp/vconn* /tmp/vlogs* /tmp/*.out /tmp/*.log
*** Removing old X11 tunnels
*** Removing excess kernel datapaths
ps ax | egrep -o 'dp[0-9]+' | sed 's/dp/nl:/'
***  Removing OVS datapaths
ovs-vsctl --timeout=1 list-br
ovs-vsctl --timeout=1 list-br
*** Removing all links of the pattern foo-ethX
ip link show | egrep -o '([-_.[:alnum:]]+-eth[[:digit:]]+)'
ip link show
*** Killing stale mininet node processes
pkill -9 -f mininet:
*** Shutting down stale tunnels
pkill -9 -f Tunnel=Ethernet
pkill -9 -f .ssh/mn
rm -f ~/.ssh/mn/*
*** Cleanup complete.
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP4$
```

FIGURE 2 – Nettoyage initial avec `sudo mn -c`

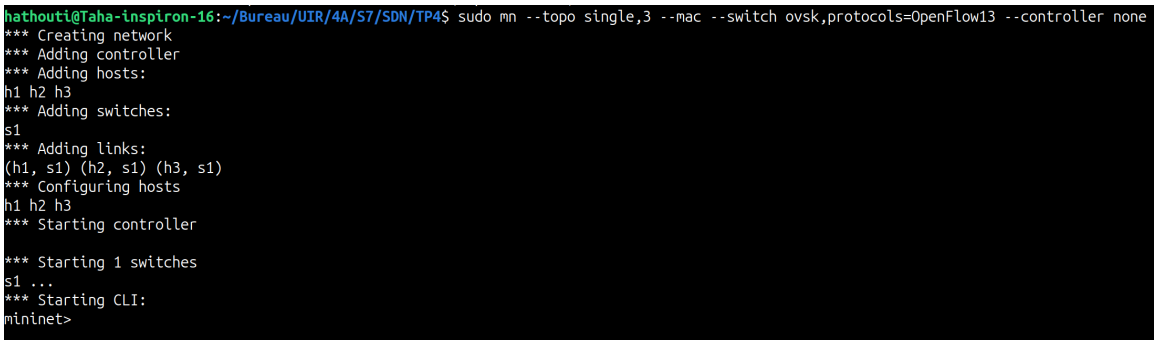


FIGURE 3 – Création de la topologie `single,3` avec `ovsk` en OpenFlow 1.3 et sans contrôleur

Observation. `sudo mn -c` se termine par `Cleanup complete`. Mininet crée ensuite les hôtes `h1 h2 h3`, le switch `s1` et les liens dans l'ordre `(h1, s1) (h2, s1) (h3, s1)` : c'est cet ordre qui attribue les ports 1, 2 et 3 du switch. Mininet affiche « Adding controller » et « Starting controller », mais la ligne qui suit est vide et aucun contrôleur n'est lancé, ce qui est l'effet de `--controller none`. L'invite `mininet>` apparaît.

Listing 2 – Terminal 2 : mode de panne, table vide, ports OpenFlow

```
sudo ovs-vsctl set-fail-mode s1 secure
sudo ovs-ofctl -O OpenFlow13 del-flows s1
sudo ovs-ofctl -O OpenFlow13 show s1
```



FIGURE 4 – Commande `sudo ovs-vsctl set-fail-mode s1 secure`



FIGURE 5 – Commande `sudo ovs-ofctl -O OpenFlow13 del-flows s1`

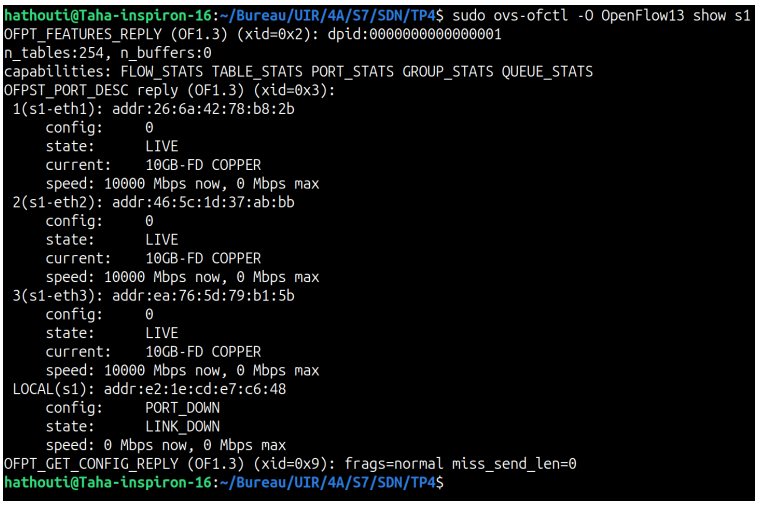


FIGURE 6 – Sortie de `sudo ovs-ofctl -O OpenFlow13 show s1`

Observation. Les deux premières commandes ne produisent aucune sortie, ce qui indique qu'elles ont réussi. `show s1` répond en OpenFlow 1.3 (`OFPT_FEATURES_REPLY (OF1.3)`) : `dpid:0000000000000001`, `n_tables:254`, et les capacités `FLOW_STATS`, `TABLE_STATS`, `PORT_STATS`, `GROUP_STATS`, `QUEUE_STATS`. Les numéros de ports sont ceux attendus :

Port OpenFlow	Interface	Adresse MAC du port	État
1	`s1-eth1`	26 :6a :42 :78 :b8 :2b	`LIVE`, 10 Gbit/s
2	`s1-eth2`	46 :5c :1d :37 :ab :bb	`LIVE`, 10 Gbit/s
3	`s1-eth3`	ea :76 :5d :79 :b1 :5b	`LIVE`, 10 Gbit/s
`LOCAL`	`s1`	e2 :1e :cd :e7 :c6 :48	`PORT_DOWN`, `LINK_DOWN`

TABLE 3 – Ports du switch `s1` d'après `show s1`

Le port `LOCAL` est l'interface interne du bridge, sans lien : il n'intervient pas dans le TP. La dernière ligne (`frags=normal miss_send_len=0`) indique que le switch n'envoie aucun paquet à un contrôleur lorsqu'aucune règle ne correspond.

Question : pourquoi utiliser le mode `secure` ?

En mode `standalone`, lorsque le switch n'a pas de contrôleur, Open vSwitch peut fonctionner comme un switch Ethernet classique (apprentissage MAC et transfert automatiques) : les pings réussiraient sans qu'aucune règle ne soit installée, et le TP ne démontrerait rien. Le mode `secure` supprime ce comportement : seuls les flux que nous installons servent au transfert. Un paquet qui ne correspond à aucune règle est alors abandonné.

2 Exemple 1 — Deux flux pour faire communiquer h1 et h2

Objectif : autoriser `h1` à communiquer avec `h2` et laisser `h3` isolé.

Listing 3 – Deux règles basées sur le port d'entrée

```
sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=100,in_port=1,
   actions=output:2"
sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=100,in_port=2,
   actions=output:1"
```

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP4$ sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=100,in_port=1,actions=output:2"
```

FIGURE 7 – Ajout de la règle `in_port=1` → `output:2`

```
hathouti@Taha-inspiron-16:~/Bureau/UIR/4A/S7/SDN/TP4$ sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=100,in_port=2,actions=output:1"
```

FIGURE 8 – Ajout de la règle `in_port=2` → `output:1`

Champ	Signification
`priority=100`	Priorité de la règle
`in_port=1`	Le paquet arrive sur le port 1
`actions=output:2`	Le paquet est envoyé sur le port 2
`in_port=2`	Le paquet arrive sur le port 2
`actions=output:1`	Le paquet est envoyé sur le port 1

TABLE 4 – Explication des deux règles

Listing 4 – Vérification dans Mininet

```
mininet> h1 ping -c 3 h2
mininet> h1 ping -c 3 h3
```

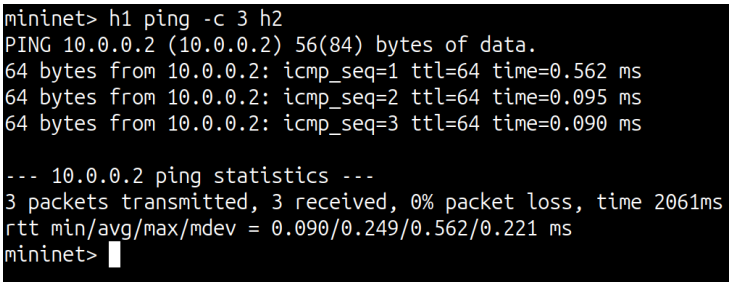


FIGURE 9 – Sortie de `h1 ping -c 3 h2`

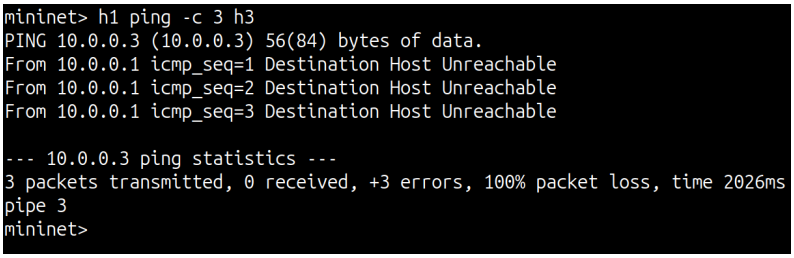


FIGURE 10 – Sortie de `h1 ping -c 3 h3`

Observation. Le ping `h1` → `h2` réussit : 3 paquets émis, 3 reçus, 0 % de perte, RTT min/moyen/max = 0,090/0,249/0,562 ms. Le premier paquet (0,562 ms) est nettement plus lent que les suivants (0,095 et 0,090 ms) : il inclut la résolution ARP. Le ping `h1` → `h3` échoue : 3 paquets émis, 0 reçu, `+3 errors`, 100 % de perte, avec le message `From 10.0.0.1 icmp_seq=1 Destination Host Unreachable` (idem pour `icmp_seq=2` et 3).

Question : pourquoi `h1` atteint-il `h2` et pas `h3` ? Pourquoi l'erreur `Destination Host Unreachable` ?

Les règles ne portent que sur le port d'entrée. Tout ce qui arrive sur le port 1 (ARP comme IPv4, puisqu'aucun protocole n'est précisé) sort sur le port 2, et inversement. Quand `h1` veut joindre `h3`, il envoie d'abord une requête ARP en diffusion pour connaître l'adresse MAC de `10.0.0.3`. Cette requête entre par le port 1 et n'est donc transférée que vers le port 2 : `h3` ne la reçoit jamais et ne peut pas répondre. `h1` ne résout pas l'adresse,

n'envoie aucun paquet ICMP et son propre noyau déclare l'échec : c'est pourquoi le message provient de `10.0.0.1` lui-même et non d'un équipement intermédiaire. Aucun paquet n'entre sur le port 3 qui corresponde à une règle : `h3` est isolé.

Remarque. Il ne s'agit pas d'un apprentissage MAC : le switch ne consulte pas l'adresse MAC de destination, il applique directement une correspondance entre ports.

3 Exemple 2 — Autoriser uniquement ICMP entre h1 et h2

Objectif : ne permettre que le protocole ICMP entre `h1` et `h2`, en autorisant ARP pour la résolution des adresses MAC.

3.1 Étapes 1 à 4 : installation des règles

Listing 5 – Réinitialisation puis règles ARP, ICMP et blocage

```
sudo ovs-ofctl -O OpenFlow13 del-flows s1
sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=200,arp,
   actions=FLOOD"
sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=100,in_port=1,
   icmp,nw_src=10.0.0.1,nw_dst=10.0.0.2,actions=output:2"
sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=100,in_port=2,
   icmp,nw_src=10.0.0.2,nw_dst=10.0.0.1,actions=output:1"
sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=0,actions=drop
   "
```



FIGURE 11 – Étape 1 : suppression des anciennes règles

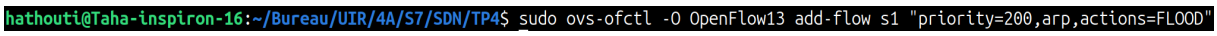


FIGURE 12 – Étape 2 : autorisation d'ARP (`priority=200`, `FLOOD`)

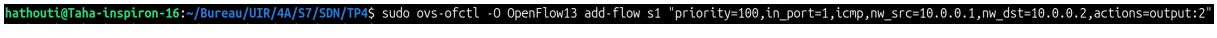


FIGURE 13 – Étape 3 : ICMP autorisé de `h1` vers `h2`

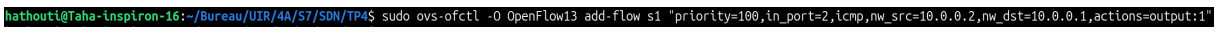


FIGURE 14 – Étape 3 : ICMP autorisé de `h2` vers `h1`

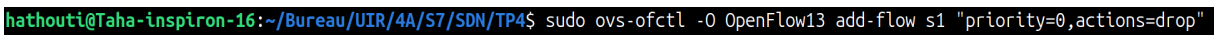


FIGURE 15 – Étape 4 : blocage explicite de tout le reste (`priority=0`, `drop`)

Priorité	Match	Action
200	ARP	`FLOOD`
100	Entrée 1, ICMP, `h1` → `h2`	`output:2`
100	Entrée 2, ICMP, `h2` → `h1`	`output:1`
0	Tous les autres paquets	`drop`

TABLE 5 – Table de flux installée à l'exemple 2

3.2 Étape 5 : tests

Listing 6 – Tests dans Mininet

```
mininet> h1 ping -c 3 h2
mininet> h1 ping -c 3 h3
```

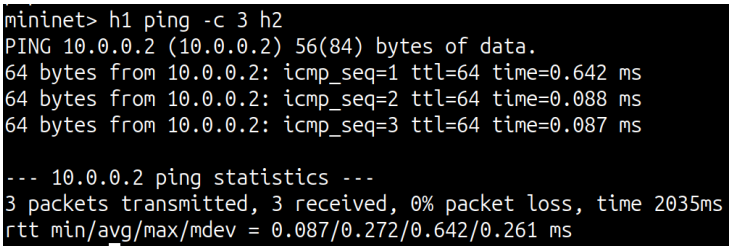


FIGURE 16 – Sortie de `h1 ping -c 3 h2`

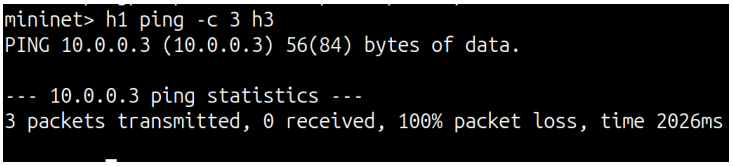


FIGURE 17 – Sortie de `h1 ping -c 3 h3`

Observation. `h1` → `h2` réussit : 3/3 reçus, 0 % de perte, RTT min/moyen/max = 0,087/0,272/0,642 ms (premier paquet à 0,642 ms à cause d'ARP). `h1` → `h3` échoue : 3 émis, 0 reçu, 100 % de perte, et cette fois **sans** message `Destination Host Unreachable` ni `+3 errors`.

Question : pourquoi l'échec de `h1` → `h3` n'a-t-il pas le même aspect qu'à l'exemple 1 ?

À l'exemple 1, la requête ARP n'atteignait pas `h3`, donc `h1` abandonnait avant d'envoyer le moindre ICMP. Ici la règle `arp,actions=FLOOD` diffuse la requête sur tous les ports sauf le port d'entrée : `h3` répond, `h1` connaît l'adresse MAC de `10.0.0.3` et envoie réellement ses trois Echo Request. C'est le switch qui les supprime : ils entrent sur le port 1 mais ne correspondent à aucune règle ICMP (la règle `h1` → `h2` exige `nw_dst=10.0.0.2`), et tombent sur la règle `priority=0,actions=drop`. `h1` attend alors des réponses qui ne viennent jamais, d'où la simple perte de 100 %.

Question : pourquoi autoriser ARP ?

Avant d'envoyer son premier paquet ICMP à `h2`, `h1` doit connaître l'adresse MAC de `h2`. Le déroulement d'un ping est donc :

Étape	Paquet	Règle qui le transfère
1	ARP Request : « qui a 10.0.0.2 ? » (diffusion)	Règle ARP (priorité 200, `FLOOD`)
2	ARP Reply : « 10.0.0.2 est à la MAC de `h2` »	Règle ARP (priorité 200, `FLOOD`)
3	ICMP Echo Request `h1` → `h2`	Règle ICMP `h1` → `h2` (priorité 100, `output:2`)
4	ICMP Echo Reply `h2` → `h1`	Règle ICMP `h2` → `h1` (priorité 100, `output:1`)

TABLE 6 – Analyse technique d'un ping et règle utilisée à chaque étape

Sans la règle ARP, la règle ICMP ne servirait à rien : la résolution d'adresse échouerait avant l'envoi du premier Echo Request. Dans cette petite topologie sans boucle, `FLOOD` facilite la démonstration ; dans un réseau de production, on préférerait des règles ARP plus restrictives.

Question : pourquoi deux règles ICMP ?

Une règle OpenFlow classique est unidirectionnelle : autoriser le paquet aller n'autorise pas automatiquement le paquet retour. Pour recevoir l'Echo Reply, il faut la règle inverse `in_port=2` → `output:1`.

4 Exemple 3 — Modifier puis supprimer un flux

On reprend la table de l'exemple 2 pour changer dynamiquement la politique du switch.

4.1 Visualiser les flux

Listing 7 – Affichage de la table de flux

```
sudo ovs-ofctl -O OpenFlow13 dump-flows s1
```

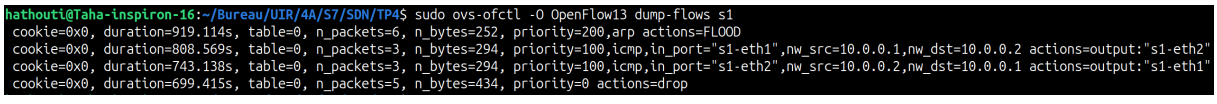


FIGURE 18 – Sortie de `sudo ovs-ofctl -O OpenFlow13 dump-flows s1`

Observation. La table contient les quatre règles de l'exemple 2, classées par priorité décroissante, avec les compteurs suivants :

Priorité	Règle	duration	n_packets	n_bytes
200	`arp` → `FLOOD`	919,114 s	6	252
100	ICMP `h1` → `h2`, sortie port 2	808,569 s	3	294
100	ICMP `h2` → `h1`, sortie port 1	743,138 s	3	294
0	`drop`	699,415 s	5	434

TABLE 7 – Compteurs relevés dans `dump-flows`

Les compteurs sont cohérents avec les tests de l'exemple 2 :

— les 6 paquets ARP font $6 \times 42 = 252$ octets (une trame ARP mesure 42 octets) ;

— chaque règle ICMP a traité 3 paquets de 98 octets ($3 \times 98 = 294$) : 98 octets = 14 (Ethernet) +84 (IP, soit 20 d'en-tête +8 ICMP +56 de données), la taille d'un Echo du `ping` par défaut. Les trois Echo Request de la règle `h1` → `h2` et les trois Echo Reply de la règle inverse correspondent aux trois pings réussis ;

— la règle `drop` a supprimé 5 paquets (434 octets), dont les trois Echo Request `h1` → `h3` ($3 \times 98 = 294$ octets). Les 2 autres paquets (140 octets) ne sont pas identifiés par cette sortie.

Les `duration` décroissent de la première à la dernière règle, dans l'ordre où elles ont été installées. `dump-flows` affiche les ports par leur nom (`"s1-eth1"`, `"s1-eth2"`) au lieu de leur numéro.

Champ	Signification
`table=0`	Table OpenFlow numéro 0
`priority`	Priorité de correspondance
`duration`	Temps écoulé depuis l'installation
`n_packets`	Nombre de paquets ayant correspondu
`n_bytes`	Nombre d'octets correspondants
`actions`	Traitement appliqué aux paquets

TABLE 8 – Champs utiles de `dump-flows`

4.2 Modifier une règle pour bloquer ICMP

Listing 8 – Remplacement de `output:2` par `drop`

```
sudo ovs-ofctl -O OpenFlow13 --strict mod-flows s1 "priority=100,
   in_port=1,icmp,nw_src=10.0.0.1,nw_dst=10.0.0.2,actions=drop"
```

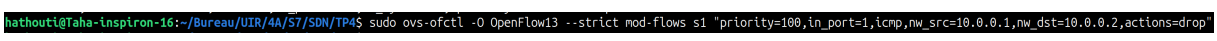


FIGURE 19 – Modification de la règle `h1` → `h2` : `actions=drop`

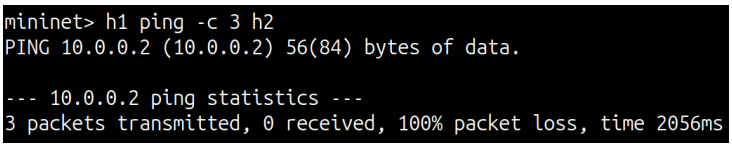


FIGURE 20 – Sortie de `h1 ping -c 3 h2` après la modification

Observation. Après la modification, le ping `h1` → `h2` échoue : 3 émis, 0 reçu, 100 % de perte (2056 ms), sans message `Destination Host Unreachable`. L'option `--strict` impose une correspondance exacte avec les champs de la règle, notamment sa priorité : seule la règle visée est modifiée. Les paquets ARP circulent toujours (règle `priority=200`), donc `h1` connaît l'adresse MAC de `h2` et envoie ses Echo Request, mais le switch les supprime.

4.3 Rétablir la communication

Listing 9 – Retour à `output:2`

```
sudo ovs-ofctl -O OpenFlow13 --strict mod-flows s1 "priority=100,
   in_port=1,icmp,nw_src=10.0.0.1,nw_dst=10.0.0.2,actions=output:2
   "
```

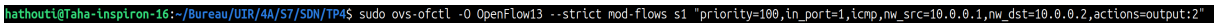


FIGURE 21 – Rétablissement de la règle `h1` → `h2` avec `actions=output:2`

La même commande `mod-flows`, avec `actions=output:2`, remet l'action d'origine : le ping `h1` → `h2` doit à nouveau fonctionner.

4.4 Supprimer uniquement cette règle

Listing 10 – Suppression ciblée de la règle `h1` → `h2`

```
sudo ovs-ofctl -O OpenFlow13 --strict del-flows s1 "priority=100,
   in_port=1,icmp,nw_src=10.0.0.1,nw_dst=10.0.0.2"
```

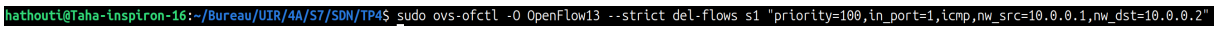


FIGURE 22 – Suppression de la seule règle `h1` → `h2`

Question : pourquoi le ping ne fonctionne-t-il plus alors que la règle inverse `h2` → `h1` est toujours installée ?

Avec `--strict`, `del-flows` ne retire que la règle dont la priorité et les champs sont exactement ceux indiqués ; les trois autres restent en place (ARP, `h2` → `h1` et `drop`). Mais un ping a besoin des deux sens : les Echo Request `h1` → `h2` n'ont plus de règle qui les autorise, ils tombent sur `priority=0,actions=drop`, et `h2` ne reçoit donc rien à qui répondre. La règle inverse existe encore mais ne sert plus.

5 Exemple 4 — Réaliser une ACL avec OpenFlow

Scénario : un administrateur autorise les communications entre les machines, sauf le trafic IP de `h1` vers `h3`.

5.1 Étapes 1 et 2 : réinitialisation et règle de base

Listing 11 – Table vide puis commutation Ethernet classique

```
sudo ovs-ofctl -O OpenFlow13 del-flows s1
sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=0,actions=
   NORMAL"
```



FIGURE 23 – Réinitialisation de la table de flux

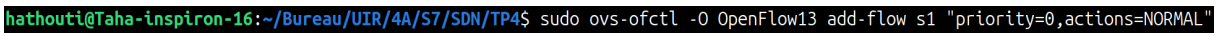


FIGURE 24 – Règle de base `priority=0,actions=NORMAL`

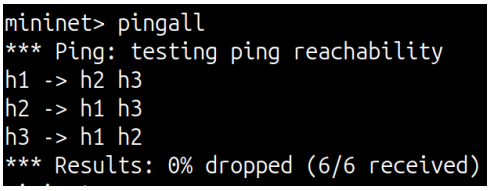


FIGURE 25 – Sortie de `pingall`

Observation. `pingall` teste les $3 \times 2 = 6$ paires : `0% dropped (6/6 received)`. Toutes les machines communiquent.

Question : que fait l'action `NORMAL`, et en quoi diffère-t-elle de `output:2` ?

`NORMAL` demande à Open vSwitch d'appliquer sa commutation Ethernet traditionnelle : apprentissage des adresses MAC, transfert vers le bon port, diffusion des trames inconnues. Ce n'est pas un `output` vers un port fixe : le port de sortie est choisi pour chaque trame. Cette règle remplace ici tout le comportement du switch en mode `standalone`, mais sous forme de règle, ce qui permet de la combiner à des règles plus précises.

5.2 Étape 3 : règle de blocage prioritaire

Listing 12 – Blocage du trafic IP de `h1` vers `h3`

```
sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=200,in_port=1,
   ip,nw_src=10.0.0.1,nw_dst=10.0.0.3,actions=drop"
```



FIGURE 26 – Ajout de la règle `priority=200` qui supprime l'IP `h1` → `h3`

Priorité	Match	Action
200	IP `h1` → `h3`, port 1	`drop`
0	Tous les autres paquets	`NORMAL`

Table 9 – Table de flux de l'ACL

5.3 Vérification

Listing 13 – Tests dans Mininet

```
mininet> h1 ping -c 3 h3
mininet> h1 ping -c 3 h2
mininet> h2 ping -c 3 h3
```

```
mininet> h1 ping -c 3 h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.

--- 10.0.0.3 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2064ms
```

Figure 27 – Sortie de `h1 ping -c 3 h3`

```
mininet> h1 ping -c 3 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.315 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.093 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.056 ms

--- 10.0.0.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2044ms
rtt min/avg/max/mdev = 0.056/0.154/0.315/0.114 ms
```

Figure 28 – Sortie de `h1 ping -c 3 h2`

```
mininet> h2 ping -c 3 h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=0.632 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=0.082 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=0.087 ms

--- 10.0.0.3 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2052ms
rtt min/avg/max/mdev = 0.082/0.267/0.632/0.258 ms
```

Figure 29 – Sortie de `h2 ping -c 3 h3`

Observation.

— `h1` → `h3` : 3 émis, 0 reçu, 100 % de perte (2064 ms) ;

— `h1` → `h2` : 3/3 reçus, RTT min/moyen/max = 0,056/0,154/0,315 ms ;

— `h2` → `h3` : 3/3 reçus, RTT min/moyen/max = 0,082/0,267/0,632 ms.

Seul le trafic `h1` → `h3` est bloqué.

Question : pourquoi la règle `drop` l'emporte-t-elle sur `NORMAL` ?

Un paquet `h1` → `h3` correspond aux deux règles : à la règle générale (qui correspond à tout) et à la règle de blocage. OpenFlow exécute les actions de la règle correspondante de **priorité la plus élevée**. La règle `drop` a la priorité 200, contre 0 pour `NORMAL` : elle gagne.

Question : pourquoi l'échec n'affiche-t-il pas `Destination Host Unreachable` ?

La règle de blocage porte sur `ip`, c'est-à-dire sur les paquets IPv4. Les paquets ARP ne sont pas de l'IPv4 : ils ne correspondent pas à cette règle et sont traités par `NORMAL`. `h1` résout donc l'adresse MAC de `h3`, émet ses Echo Request, qui sont supprimés, d'où une simple perte de 100 %.

Lien avec les ACL Cisco. Cette table se lit comme une ACL : la règle `priority=200` joue le rôle d'une ligne `deny ip host 10.0.0.1 host 10.0.0.3` et la règle `priority=0,actions=NORMAL` celui d'une ligne finale `permit ip any any`. La différence est que l'ordre d'évaluation n'est pas l'ordre des lignes mais la priorité des règles.

Limite. Cette ACL est *stateless* : elle examine chaque paquet isolément, sans suivi de session. Elle bloque les paquets IP `h1` → `h3` mais ne constitue pas, à elle seule, un pare-feu à états.

6 Exemple 5 — Autoriser uniquement un service HTTP sur le port TCP 8000

Scénario : `h2` héberge un serveur HTTP. On autorise `h1` à y accéder sur le port TCP 8000, et aucun autre trafic IPv4.

6.1 Installation des règles

Listing 14 – Règles de l'exemple 5

```
sudo ovs-ofctl -O OpenFlow13 del-flows s1
sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=200,arp,
   actions=FLOOD"
sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=100,in_port=1,
   tcp,nw_src=10.0.0.1,nw_dst=10.0.0.2,tp_dst=8000,actions=output
   :2"
sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=100,in_port=2,
   tcp,nw_src=10.0.0.2,nw_dst=10.0.0.1,tp_src=8000,actions=output
   :1"
sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=0,actions=drop
   "
```



Figure 30 – Réinitialisation de la table de flux

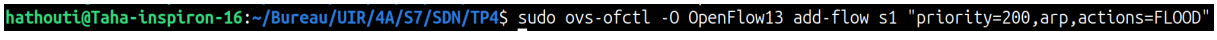


Figure 31 – Autorisation d'ARP

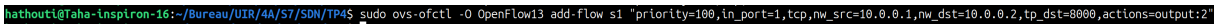


FIGURE 32 – Autorisation des requêtes TCP `h1` → `h2` (`tp_dst=8000`)

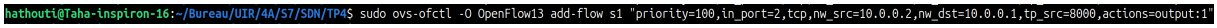


FIGURE 33 – Autorisation des réponses TCP `h2` → `h1` (`tp_src=8000`)



FIGURE 34 – Blocage de tous les autres paquets

Question : pourquoi deux règles TCP ?

Direction	Champ TCP examiné	Action
`h1` → `h2`	Destination = 8000 (`tp_dst`)	`output:2`
`h2` → `h1`	Source = 8000 (`tp_src`)	`output:1`

TABLE 10 – Les deux règles TCP

Le client choisit généralement un port source éphémère ; le serveur répond depuis le port 8000 vers ce port éphémère. Les requêtes se reconnaissent donc au port de destination 8000, et les réponses au port source 8000. Une règle unique ne pourrait pas couvrir les deux sens, pour la même raison que les deux règles ICMP de l'exemple 2.

6.2 Démarrage du serveur et tests

Listing 15 – Serveur HTTP sur `h2`, requête depuis `h1`, puis ping

```
mininet> h2 python3 -m http.server 8000 --bind 10.0.0.2 &
mininet> h1 curl http://10.0.0.2:8000
mininet> h1 ping -c 3 h2
```

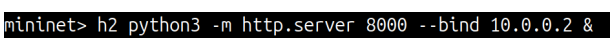


FIGURE 35 – Lancement du serveur HTTP sur `h2`

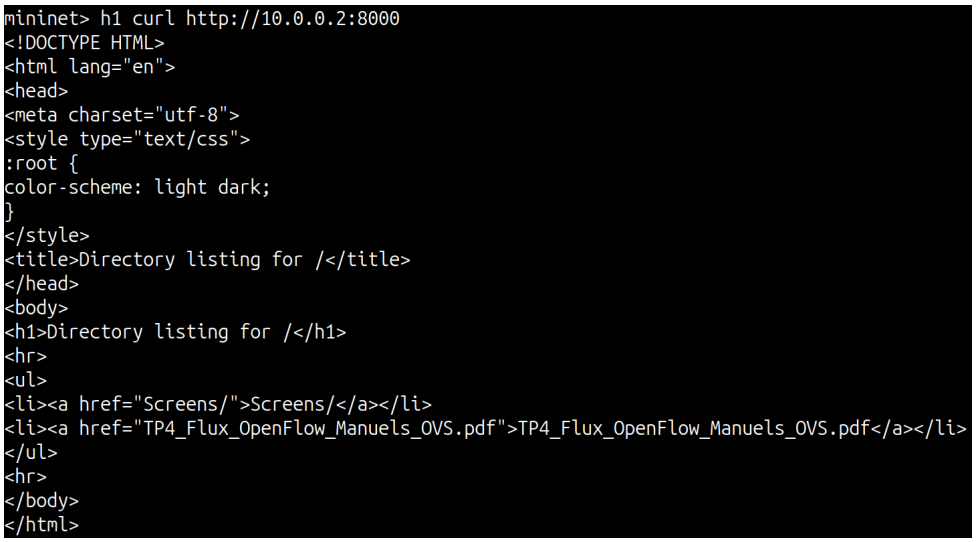


FIGURE 36 – Sortie de `h1 curl http://10.0.0.2:8000`

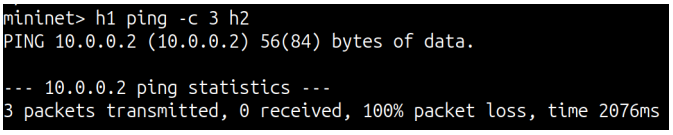


FIGURE 37 – Sortie de `h1 ping -c 3 h2`

Observation. Le `curl` réussit : `h1` récupère une page HTML intitulée `Directory listing for /`, qui liste le dossier depuis lequel Mininet a été lancé (`Screens/` et `TP4_Flux_OpenFlow_Manuels_OVS.pdf`). Côté serveur, la console Mininet affiche `Serving HTTP on 10.0.0.2 port 8000` puis la ligne de journal `10.0.0.1 - - [20/Oct/2026 22:35:10] "GET / HTTP/1.1" 200 -` : la requête de `h1` est bien arrivée sur `h2`. Le ping `h1` → `h2`, lui, échoue : 3 émis, 0 reçu, 100 % de perte (2076 ms).

Interprétation. Les paquets ICMP ne correspondent à aucune règle TCP : ils tombent sur `priority=0,actions=drop`. Une politique OpenFlow peut donc être précise jusqu'aux champs de transport TCP/UDP, contrairement à l'exemple 1 qui ne considérait que les ports physiques. L'ARP reste autorisé (`FLOOD`), ce qui permet à la connexion TCP de s'établir.

Limite. Ces deux règles ne réalisent pas un véritable suivi de connexion TCP : tout segment `h1` → `h2` avec le port de destination 8000, et tout segment `h2` → `h1` avec le port source 8000, est transféré, qu'il appartienne ou non à une connexion établie. Un filtrage à états nécessiterait des mécanismes supplémentaires, comme `conntrack`.

7 Exemple 6 — Gérer la durée de vie des flux

Dans les exemples précédents, les règles restent installées tant qu'elles ne sont pas supprimées. OpenFlow permet aussi de définir des délais d'expiration.

Listing 16 – Règle avec `idle_timeout=30`

```
sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=150,icmp,
   nw_src=10.0.0.1,nw_dst=10.0.0.2,idle_timeout=30,actions=output
   :2"
```

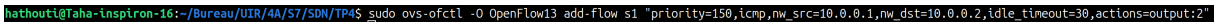


FIGURE 38 – Ajout d'un flux ICMP `h1` → `h2` avec `idle_timeout=30`

Observation. La commande s'exécute sans erreur. `idle_timeout=30` n'est pas une commande à part : c'est un paramètre placé dans la chaîne de la règle, à côté de `priority` et de `actions`. La règle expire après 30 secondes sans paquet correspondant, et le compteur d'inactivité est remis à zéro chaque fois qu'un paquet correspond à la règle.

Paramètre	Comportement
`idle_timeout=30`	Suppression après 30 s d'inactivité
`hard_timeout=60`	Suppression après 60 s de durée de vie
`idle_timeout=0`	Pas d'expiration par inactivité
`hard_timeout=0`	Pas d'expiration liée à la durée totale

TABLE 11 – Paramètres d'expiration d'un flux

Question : quelle différence entre `idle_timeout` et `hard_timeout` ?
Avec `idle_timeout=30`, la règle disparaît seulement si elle ne reçoit aucun paquet pendant 30 s : un trafic régulier la maintient en place. Avec `hard_timeout=60`, la règle disparaît 60 s après son installation, même si elle reçoit du trafic en continu. Le premier convient à un flux qui doit durer tant qu'il est utilisé, le second à une autorisation limitée dans le temps.

Identifier les flux avec un cookie. Le `cookie` est une étiquette numérique attachée à une règle :

Listing 17 – Règle étiquetée par un cookie, puis suppression de tous les flux portant ce cookie (commandes du sujet)

```
sudo ovs-ofctl -O OpenFlow13 add-flow s1 "cookie=0x42,priority
   =150,icmp,nw_src=10.0.0.1,nw_dst=10.0.0.2,actions=output:2"
sudo ovs-ofctl -O OpenFlow13 del-flows s1 "cookie=0x42/-1"
```

La seconde commande supprime uniquement les flux dont le cookie vaut `0x42` (le masque `/-1` impose la comparaison de tous les bits). Cette méthode est utile pour gérer plusieurs politiques SDN indépendantes : chaque politique reçoit son cookie et peut être retirée sans toucher aux autres.

8 Synthèse des commandes OVS

Commande	Fonction
`ovs-ofctl show s1`	Afficher les ports OpenFlow
`ovs-ofctl dump-flows s1`	Consulter la table de flux
`ovs-ofctl add-flow`	Ajouter une règle
`ovs-ofctl mod-flows`	Modifier une règle
`ovs-ofctl del-flows`	Supprimer des règles
`ovs-ofctl dump-ports s1`	Examiner les statistiques des ports
`ovs-vsctl show`	Visualiser la configuration OVS

TABLE 12 – Commandes Open vSwitch utilisées ou citées dans le TP (avec `-O OpenFlow13` pour les commandes `ovs-ofctl`)

Exemple	Politique	Critères de match	Résultat observé
1	`h1` ↔ `h2`, `h3` isolé	Port d'entrée	`h1` → `h2` 0 % de perte ; `h1` → `h3` `Host Unreachable`
2	ICMP seulement entre `h1` et `h2`	ARP, ICMP, adresses IP	`h1` → `h2` OK ; `h1` → `h3` 100 % de perte
3	Modification et suppression d'un flux	`--strict` (priorité + champs)	Ping bloqué après `drop`
4	ACL : tout autorisé sauf IP `h1` → `h3`	IPv4, priorité 200 *vs* 0	`pingall` 6/6 avant ; seul `h1` → `h3` bloqué après
5	Service TCP 8000 uniquement	TCP, `tp_dst`, `tp_src`	`curl` OK ; ping 100 % de perte
6	Durée de vie et cookie	`idle_timeout`, `hard_timeout`, `cookie`	Règle acceptée par `add-flow`

TABLE 13 – Récapitulatif des six ateliers

9 Observation avec Wireshark

La capture se fait sur `s1-eth1` ou `s1-eth2`, interfaces du switch côté `h1` et `h2`. La capture est lancée **avant** la commande Mininet qui génère le trafic, puis le filtre est saisi dans la barre d'affichage. Les interfaces `s1-ethX` n'existent que tant que Mininet tourne.

Listing 18 – Lancement de Wireshark sur `s1-eth2`

```
sudo wireshark -i s1-eth2 -k &
```

Expérience	Filtre Wireshark	Trafic généré dans Mininet
Résolution des adresses	`arp`	`h1 ip neigh flush all` puis `h1 ping -c 1 h2`
Ping autorisé ou bloqué	`icmp`	`h1 ping -c 3 h2`
Service HTTP	`tcp.port == 8000`	`h1 curl http://10.0.0.2:8000`
Établissement de connexion TCP	`tcp.flags.syn == 1`	`h1 curl http://10.0.0.2:8000`
Communication `h1` → `h2`	`ip.src == 10.0.0.1 && ip.dst == 10.0.0.2`	`h1 ping -c 3 h2`

TABLE 14 – Filtres Wireshark et trafic à générer

Observation. Une première capture sur `s1-eth2`, réalisée avec les règles de l'exemple 5, montre :

- l'échange ARP : « Who has 10.0.0.2 ? Tell 10.0.0.1 » suivi de « 10.0.0.2 is at 00 :00 :00 :00 :00 :02 », puis le même échange pour `10.0.0.3` : la règle `arp,actions=FLOOD` fonctionne ;
- trois Echo Request `10.0.0.2` → `10.0.0.3` (`seq=1`, 2, 3) marqués `no response found!` par Wireshark : les paquets sortent de `h2` mais le switch les supprime, aucune réponse ne revient ;
- quelques paquets MDNS sans rapport avec le TP, émis par le système Ubuntu lui-même.

Pourquoi capturer sur `s1-eth2` ? La capture sur l'interface de sortie permet de vérifier si les paquets arrivent réellement au port de destination. Un paquet supprimé par une règle `drop` dès son entrée sur le switch n'apparaît pas sur `s1-eth2` : en comparant `s1-eth1` et `s1-eth2`, on voit qu'il a été bloqué par le switch et non perdu ailleurs.

Messages OpenFlow. Les commandes `ovs-ofctl` utilisent localement un socket Unix : leurs messages OpenFlow ne sont normalement pas visibles dans une capture TCP classique sur l'interface loopback, et aucune manipulation supplémentaire n'est nécessaire.

10 Synthèse

Ce qu'il faut comprendre. Le fonctionnement d'OpenFlow se résume ainsi :

paquet entrant → extraction des champs → comparaison avec les règles → sélection de la règle prioritaire → exécution des actions.

Réponse à la question fil rouge. Sans contrôleur, en mode `secure`, un switch Open vSwitch ne transfère que ce que ses règles autorisent : chaque paquet est comparé à la table de flux, la règle de plus haute priorité qui correspond s'applique, et un paquet sans règle correspondante est abandonné. Les règles se construisent à la main avec `ovs-ofctl` et peuvent porter sur le port d'entrée, sur ARP, ICMP, IPv4 ou sur les ports TCP. Elles sont

unidirectionnelles (il faut une règle par sens), ordonnées par priorité (règle générale à faible priorité, exceptions à forte priorité) et peuvent être modifiées, supprimées sélectivement (`--strict`, `cookie`) ou limitées dans le temps (`idle_timeout`, `hard_timeout`). Le même switch peut ainsi réaliser du switching (`NORMAL`), du filtrage ACL, l'autorisation d'un service TCP ou l'isolation d'un hôte, sans modification matérielle.

Conclusion. À la fin du TP, je sais installer, modifier et supprimer des flux OpenFlow à la main dans Open vSwitch, expliquer pourquoi une règle gagne sur une autre grâce à la priorité, relier une table de flux à une ACL, et vérifier le comportement du switch avec les messages du ping, les compteurs de `dump-flows` et Wireshark.

11 Grille de validation rapide

Vérification	**Commande / observation**	**OK**
Topologie et ports vérifiés	`show s1` : ports 1, 2, 3 = `s1-eth1` à `s1-eth3`	☒
Mode `secure` activé	`set-fail-mode s1 secure`	☒
Exemple 1 : ports	`h1` → `h2` OK, `h1` → `h3` `Host Unreachable`	☒
Exemple 2 : ARP + ICMP + drop	`h1` → `h2` OK, `h1` → `h3` 100 % de perte	☒
Exemple 3 : `dump-flows`, `mod-flows`, `del-flows`	Compteurs lus, ping bloqué après `drop`	☒
Exemple 4 : ACL	`pingall` 6/6, seul `h1` → `h3` bloqué	☒
Exemple 5 : service TCP 8000	`curl` OK, ping 100 % de perte	☒
Exemple 6 : durée de vie et cookie	`idle_timeout=30` installé	☒
Observation Wireshark	Captures `arp`, `icmp`, `tcp.port == 8000`	☐

Table 15 – Grille de validation